



**TOR VERGATA**  
UNIVERSITÀ DEGLI STUDI DI ROMA

**2025/2026**



# **TUTORA**

---

*ISPW Project Documentation*

**Alessio Dainelli**  
**Vittorio Iozzia**

# Index

---

|                                              |           |
|----------------------------------------------|-----------|
| <b>1. SOFTWARE REQUIREMENT SPECIFICATION</b> | <b>2</b>  |
| <b>1.1 Introduction</b>                      | <b>2</b>  |
| <i>1.1.1 Aim of the document</i>             | <b>2</b>  |
| <i>1.1.2 Overview of the System</i>          | <b>2</b>  |
| <i>1.1.3 HW &amp; SW Requirements</i>        | <b>3</b>  |
| <i>1.1.4 Related Systems, Pros and Cons</i>  | <b>4</b>  |
| <b>1.2 User Stories</b>                      | <b>5</b>  |
| <b>1.3 Functional Requirements</b>           | <b>5</b>  |
| <b>1.4 Use Cases</b>                         | <b>6</b>  |
| <i>1.4.1 Overview Diagram</i>                | <b>6</b>  |
| <i>1.4.2 Internal Steps</i>                  | <b>7</b>  |
| <b>2. STORYBOARDS</b>                        | <b>9</b>  |
| <b>3. DESIGN</b>                             | <b>13</b> |
| <b>3.1 Class Diagram</b>                     | <b>13</b> |
| <i>3.1.1 Design Patterns</i>                 | <b>15</b> |
| <b>3.2 Activity Diagram</b>                  | <b>22</b> |
| <b>3.3 Sequence Diagram</b>                  | <b>25</b> |
| <b>3.4 State Diagram</b>                     | <b>27</b> |
| <b>4. TESTING</b>                            | <b>28</b> |
| <b>5. CODE</b>                               | <b>30</b> |
| <b>6. VIDEO</b>                              | <b>30</b> |
| <b>7. SONAR CLOUD</b>                        | <b>30</b> |

# 1. Software Requirement Specification

---

## 1.1 Introduction

### 1.1.1 Aim of the document

The purpose of this document is to outline the software requirements and architectural implementation of "Tutora", a private tutoring marketplace platform developed as part of the Software Engineering and Web Design course (A.Y. 2025-2026).

This document details the functional specifications and design patterns adopted to ensure a scalable and robust solution.

### 1.1.2 Overview of the System

Tutora is designed to connect students with qualified tutors, enabling students to book one-on-one tutoring sessions in a wide range of academic subjects. Tutors can apply to offer their expertise in specific subject categories, pending administrative approval, and then create and manage available lesson slots.

From a technical perspective, the system is a Java-based application engineered according to the *Model-View-Controller (MVC)* architectural pattern. The project is managed using Maven for build automation and features a *Graphical User Interface (GUI)* implemented in *JavaFX*.

The system supports the following main functionalities:

- **User Management:** Role-based authentication (Student, Tutor, Admin).
- **Expertise Management:** Tutors apply for subject expertise; Admins approve or reject applications.
- **Lesson Scheduling:** Creation, validation, and booking of tutoring sessions.
- **Payment Processing:** Secure integration for transaction management with payment status tracking (Pending, Paid, Refunded).
- **Notification System:** Real-time notifications to users on booking events, approvals, and reviews.
- **Review System:** Students can leave star ratings and comments after completed lessons.

- Session Tracking: Handling of concurrent user sessions via token-based mechanisms.

### 1.1.3 HW & SW Requirements

To ensure the proper functioning of the Tutora application for both development and end-user execution, the following environments are defined

- Software Requirements:
  - **Operating System:** Cross-platform compatibility (Windows 10/11, macOS 12+, Linux Ubuntu 20.04+).
  - **Development Kit (JDK):** Java Development Kit 21 (LTS) or higher.
  - **Build Automation:** Apache Maven 3.8+.
  - **User Interface:** JavaFX SDK 21.
  - **Database:** MySQL Community Server 8.0 or MariaDB 10.6+. Alternative persistence modes: In-Memory Demo mode and JSON-file mode for development and testing.
  - **IDE (Optional):** IntelliJ IDEA or Eclipse IDE for development.
  - **Version Control:** Git.
- Hardware Requirements:
  - **Processor:** Modern Multi-core Processor (Intel Core i5 / AMD Ryzen 5 or equivalent).
  - **RAM:** Minimum 8GB (16GB Recommended for development).
  - **Storage:** At least 500MB of free disk space for application and local database storage.
  - **Display:** Minimum resolution of 1366x768 (1920x1080 recommended for optimal GUI experience).
  - **Network:** Active internet connection required for payment gateway integration.

### **1.1.4 Related Systems, Pros and Cons**

#### **Preply**

##### *Pros:*

- Large marketplace with thousands of tutors across many languages and subjects
- Integrated video call and messaging tools
- Subscription plans available

##### *Cons:*

- High platform commission fees for tutors
- Limited filtering options for niche academic subjects
- No role for institutional approval of tutor qualifications

#### **Wyzant**

##### *Pros:*

- Strong tutor vetting process with background checks
- Flexible scheduling and in-person/online options
- Good student review system

##### *Cons:*

- Primarily US-focused, limited international availability
- High service fee (25%) on tutor earnings
- No admin/institution role for quality oversight

## 1.2 User Stories

*Vittorio Iozzia:*

1. As a student, I want to search for a tutor, so that I can find the most suitable one for my learning needs.
2. As a tutor, I want to set my availability for specific days, so that students can book lessons according to my schedule.
3. As an admin, I want to add tutoring categories, so that I can expand the platform's offerings.

*Alessio Dainelli:*

1. As a student, I want to reserve lessons, so that I can improve my skills.
2. As a tutor, I want to do lessons, so that I can earn more money.
3. As an admin, I want to check the documents of the tutor, so that I can make the application reliable.

## 1.3 Functional Requirements

*Vittorio Iozzia:*

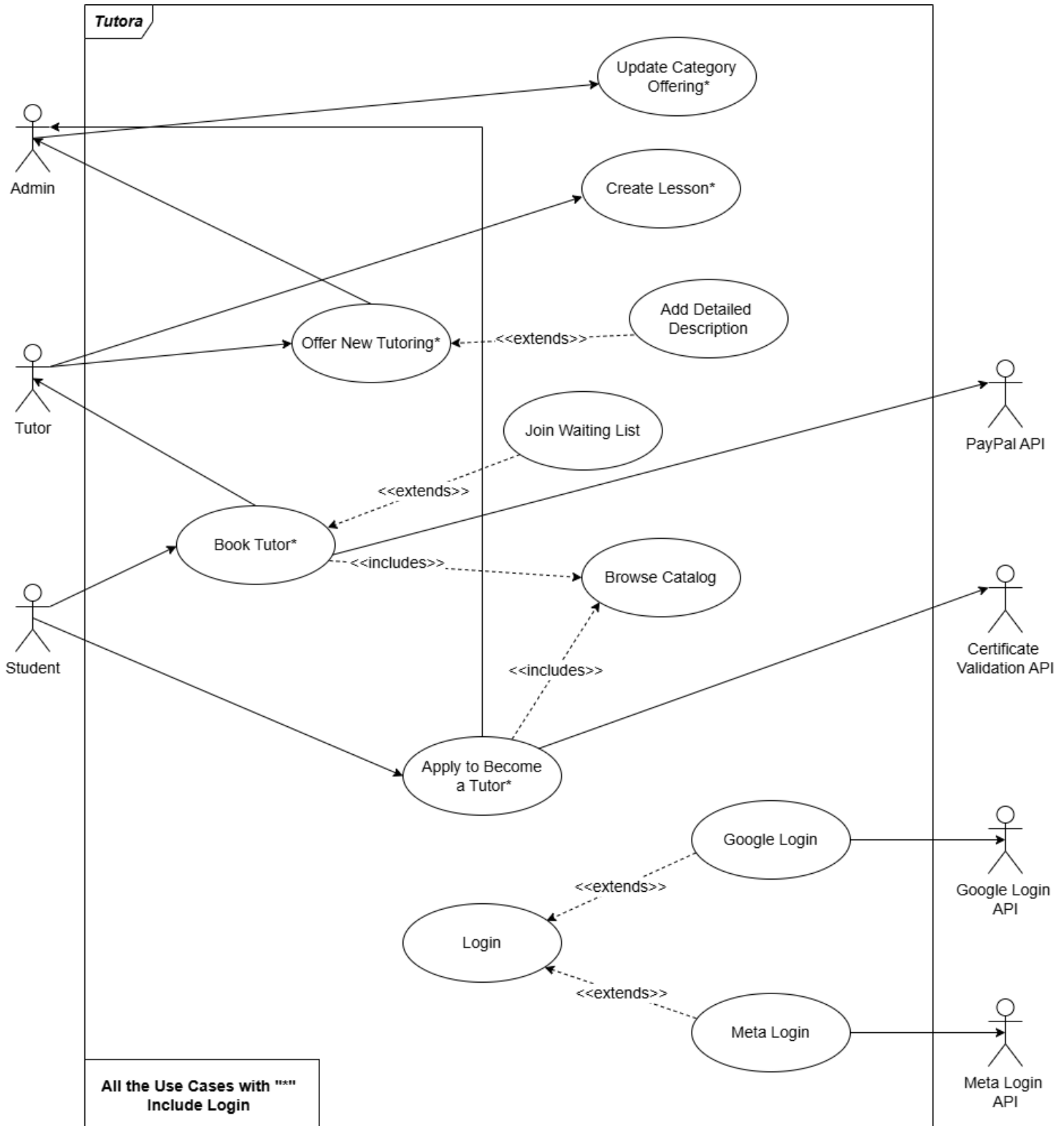
1. The system shall send a notification to the tutor when his lesson is booked with the purchaser's username.
2. The system shall provide the reservation of one or more tutor lessons.
3. The system shall store an application as evaluated when it's assessed.

*Alessio Dainelli:*

4. The system shall provide the selection of filters in tutor search.
5. The system shall check the success of lessons payment.
6. The system shall send a notification to the student when a tutor booking request is accepted or rejected.

## 1.4 Use Cases

### 1.4.1 Overview Diagram



### 1.4.2 Internal Steps

*Vittorio Iozzia:*

UC-2: Apply to Become a Tutor

#### MAIN FLOW:

1. The student selects the search filters.
2. The system gets the matching categories from the Category Catalog System.
3. The student selects a specific category.
4. The system displays the application form.
5. The student enters the requested information and documents.
6. The student submits the application.
7. The system sends a notification to the admin with the application details.
8. The admin evaluates the application.
9. The system sends a notification to the student with the evaluation result.

#### EXTENSIONS:

**2a. No matching tutoring category is found:** The system displays an error message and asks to select different filters.

**2b. The Category Catalog System doesn't respond:** The system displays an error message asking to try later and terminates the use case.

**5a. The student submits invalid documents:** The system displays an error message listing the invalid documents.

*Alessio Dainelli:*

UC-1: Book Tutor

#### MAIN FLOW:

1. The student selects a search filter.
2. The system gets the matching tutors from the Tutor Catalog System.
3. The student selects an available tutor.
4. The system sends the tutor the request with the lesson details.
5. The tutor accepted the request.
6. The system prepares a payment form.
7. The student selects the payment method and authorizes the payment.
8. The system validates the transaction with the Payment Gateway.
9. The system saves the schedule.



**EXTENSION:**

**2a. The system doesn't find lessons that match the filter:** The system shows a message and asks the student to change the filter.

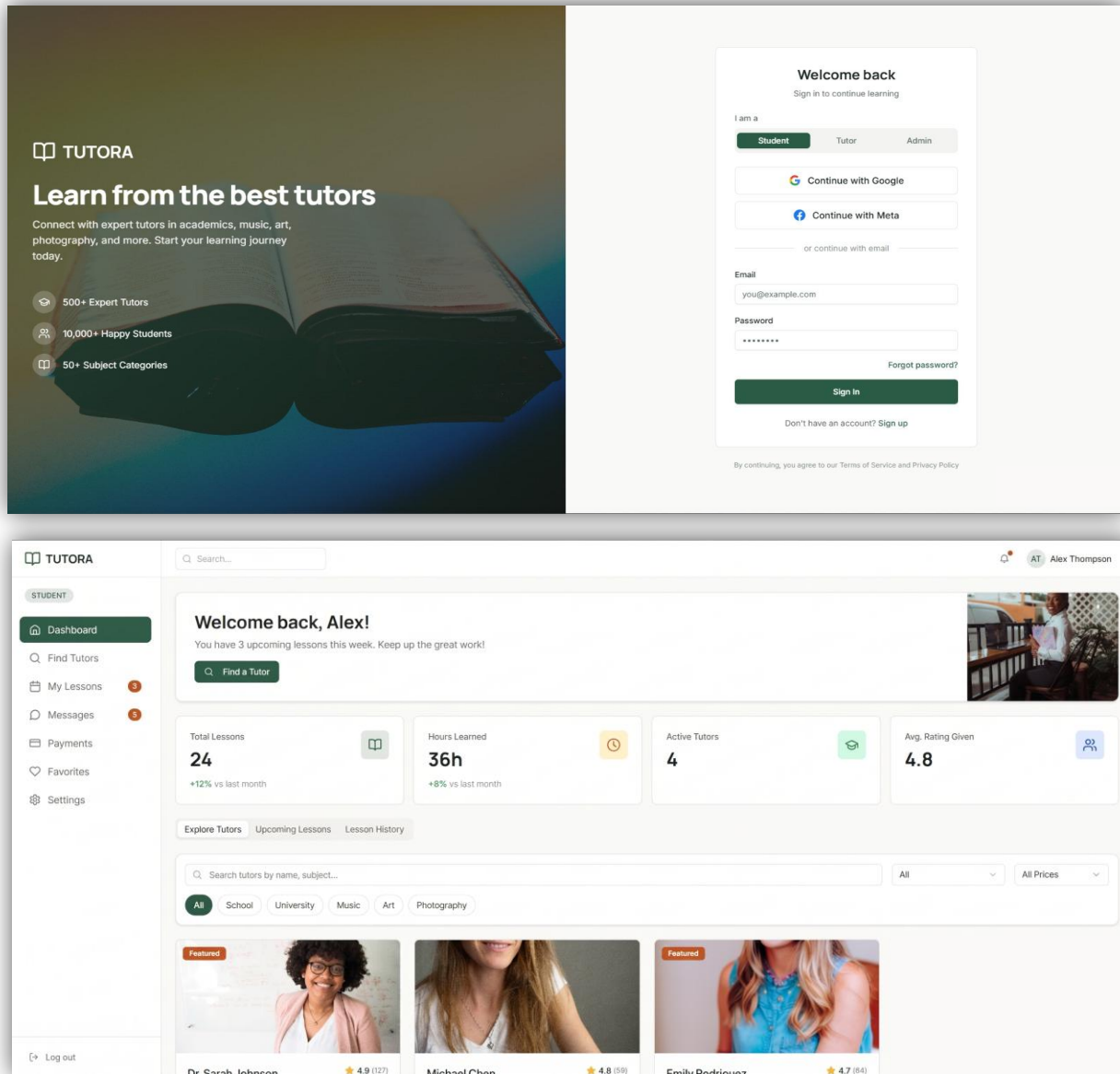
**2b. Tutor Catalog System does not respond:** The system notifies the student with an error message and terminates the use case.

**5a. Tutor does not accept the request:** The system notifies the student with a message and eliminates the request.

**8a. Payment Gateway does not respond:** The system notifies the student with an error message and request to retry the payment or change method.

## 2. Storyboards

The following section presents the project's storyboards, which were developed using Figma to visually map out the user experience. These diagrams illustrate the core interaction flow and provide a clear conceptual framework for the proposed solution.



TUTORA

STUDENT

Dashboard

Find Tutors

**My Lessons**

Messages

Payments

Favorites

Settings

Log out

Q Search...

### My Lessons

Manage your upcoming and past lessons

Upcoming (3) Past (3) Cancelled (1)

Calculus II

with Dr. Sarah Johnson

Today 3:00 PM - 4:00 PM Online

Join Lesson Reschedule Cancel

Piano Basics

with Michael Chen

Tomorrow 10:00 AM - 10:45 AM Online

Join Lesson Reschedule Cancel

Digital Photography

with Emily Rodriguez

Jan 15, 2024 2:00 PM - 3:30 PM Chicago Studio

Join Lesson Reschedule Cancel

Calendar

View your lesson schedule

< April 2026 >

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| Su | Mo | Tu | We | Th | Fr | Sa |
| 29 | 30 | 31 | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 1  | 2  |

Quick Stats

3 This week

12 This month

TUTORA

STUDENT

Dashboard

Find Tutors

My Lessons

Messages

**Payments**

Favorites

Settings

Log out

Q Search...

### Payments

Manage your payments and billing

+ Ad Funds

Account Balance

\$145.00

\$

This Month

\$210.00

Total Spent

\$1,240.00

Transaction History

Payment Methods

Transaction History

View all your past transactions

| Transaction ID | Date         | Description                           | Amount    |           |
|----------------|--------------|---------------------------------------|-----------|-----------|
| TXN-001        | Jan 12, 2024 | Calculus II - Dr. Sarah Johnson       | -\$45.00  |           |
| TXN-002        | Jan 10, 2024 | Piano Basics - Michael Chen           | -\$55.00  |           |
| TXN-003        | Jan 8, 2024  | Account Top-up                        | +\$200.00 | completed |
| TXN-004        | Jan 5, 2024  | Photography Session - Emily Rodriguez | -\$80.00  | completed |
| TXN-005        | Jan 3, 2024  | Physics Fundamentals - James Wilson   | -\$80.00  | refunded  |

This Month

This Week

This Month

This Year

All Time

Export

TUTORA

STUDENT

Dashboard

Find Tutors

My Lessons

**Messages**

Payments

Favorites

Settings

Log out

Q Search...

### Messages

Q Search conversations...

Dr. Sarah Johnson

Great progress today! Keep practicing

Michael Chen

See you tomorrow for the piano lesson

Emily Rodriguez

Don't forget to bring your camera for t

James Wilson

The physics homework solutions are i

Dr. Sarah Johnson

Online

Hi Alex! How did your practice session go?

10:30 AM

It went well! I managed to solve most of the problems.

10:32 AM

That's great to hear! Which problems gave you trouble?

10:33 AM

The integration by parts problems are still confusing me a bit.

10:35 AM

No worries, that's a common challenge. We'll focus on that in our next session. I'll prepare some extra examples for you.

10:36 AM

Thank you so much! Looking forward to it.

10:38 AM

Great progress today! Keep practicing those integrals.

11:45 AM

Type a message...

10

TUTORA

STUDENT

Dashboard

Find Tutors

My Lessons3

Messages3

Payments

Favorites

Settings

Log out

Search...

Alex Thompson

Favorite Tutors

Your saved tutors for quick access

Featured



Dr. Sarah Johnson

Mathematics Professor

Mathematics University

New York 10+ years

4.9 (127)

\$45/hour

Profile

Book



Michael Chen

Piano Instructor

Music Piano

Los Angeles 8 years

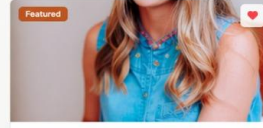
4.8 (88)

\$55/hour

Profile

Book

Featured



Emily Rodriguez

Art & Design Teacher

Art Photography

Chicago 6 years

4.7 (64)

\$40/hour

Profile

Book

TUTORA

STUDENT

Dashboard

Find Tutors

My Lessons3

Messages3

Payments

Favorites

Settings

Log out

Search...

Alex Thompson

Settings

Manage your account settings and preferences

Profile

Notifications

Security

Preferences

Preferences

Customize your experience

Theme

Light

Default Lesson Duration

45 minutes

Calendar View

Week View

TUTORA

TUTOR

Dashboard

My Schedule

Bookings4

Students

Earnings

Reviews

Messages3

Profile

Settings

Log out

Search...

Dr. Sarah Johnson

Edit Profile

Tutor Dashboard

Manage your schedule, bookings, and profile

Total Students

48

+8% vs last month

Lessons This Month

32

+12% vs last month

Earnings

\$1,440

+15% vs last month

Avg. Rating

4.9

Availability

Upcoming Lessons

Manage Availability

Set your available time slots for students to book

< April 2026 >

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| Su | Mo | Tu | We | Th | Fr | Sa |
| 29 | 30 | 31 | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 1  | 2  |

Time Slots for 4/9/2026

9:00 AM

Available

Available

10:00 AM

Available

Available

11:00 AM

Unavailable

Unavailable

12:00 PM

Unavailable

Unavailable

1:00 PM

Available

Available

Booking Requests

2 new

AT Alex Thompson

Calculus II

Jan 15, 2024 3:00 PM

Accept

Decline

pending

EW Emma Wilson

Linear Algebra

Jan 19, 2024 10:00 AM

Accept

Decline

pending

MB Michael Brown

Accept

Decline

confirmed

11

TUTORA

Search...

DS Dr. Sarah Johnson

TUTOR

Dashboard

My Schedule

Bookings

Students

Earnings

Reviews

Messages

Profile

Settings

Log out

My Profile

Manage your public profile information

Profile Preview

How students see your profile



Dr. Sarah Johnson

Mathematics Professor

4.9 (127 reviews)

Mathematics Calculus Linear Algebra

\$45/hour

View Public Profile

Edit Profile

Update your profile information



Profile Photo

A professional photo helps build trust with students

Display Name

Dr. Sarah Johnson

Professional Title

Mathematics Professor

Bio

Passionate mathematics educator with over 10 years of experience teaching at university level. I specialize in making complex concepts accessible and enjoyable for students of all levels.

Hourly Rate (\$)

45

Years of Experience

10+

Location

New York, NY

Education

Ph.D. Mathematics, MIT

Subjects & Categories

Mathematics Calculus Linear Algebra Statistics

Add a subject...

TUTORA

Search...

AU Admin User

ADMIN

Dashboard

Users

Tutors

Approvals

Analytics

Reports

Settings

Log out

Admin Dashboard

Manage users, tutors, and platform settings

Add User

Total Users

2,847

+12% vs last month

Active Tutors

234

+8% vs last month

Total Lessons

8,429

+15% vs last month

Platform Revenue

\$124,580

+22% vs last month

Tutor Approvals

User Management

Analytics

Pending Tutor Applications

Review and approve new tutor registrations

DM

David Martinez

david.m@email.com

Chemistry 5 years exp.

Details

Approve

Reject

LW

Lisa Wang

lisa.w@email.com

Piano, Music Theory 8 years exp.

Details

Approve

Reject

RK

Robert Kim

robert.k@email.com

Photography, Art 6 years exp.

Details

Approve

Reject

12

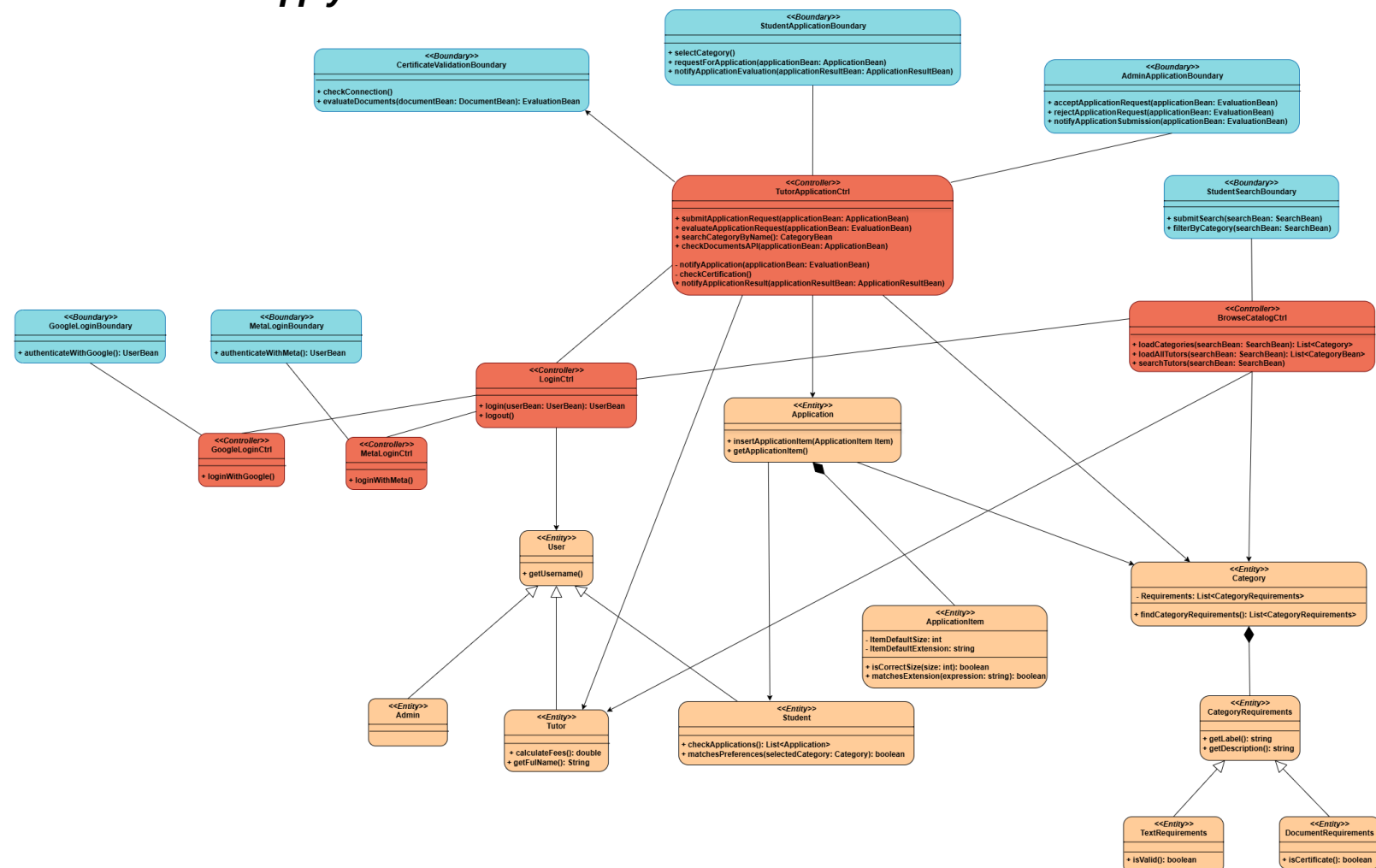
## 3. Design

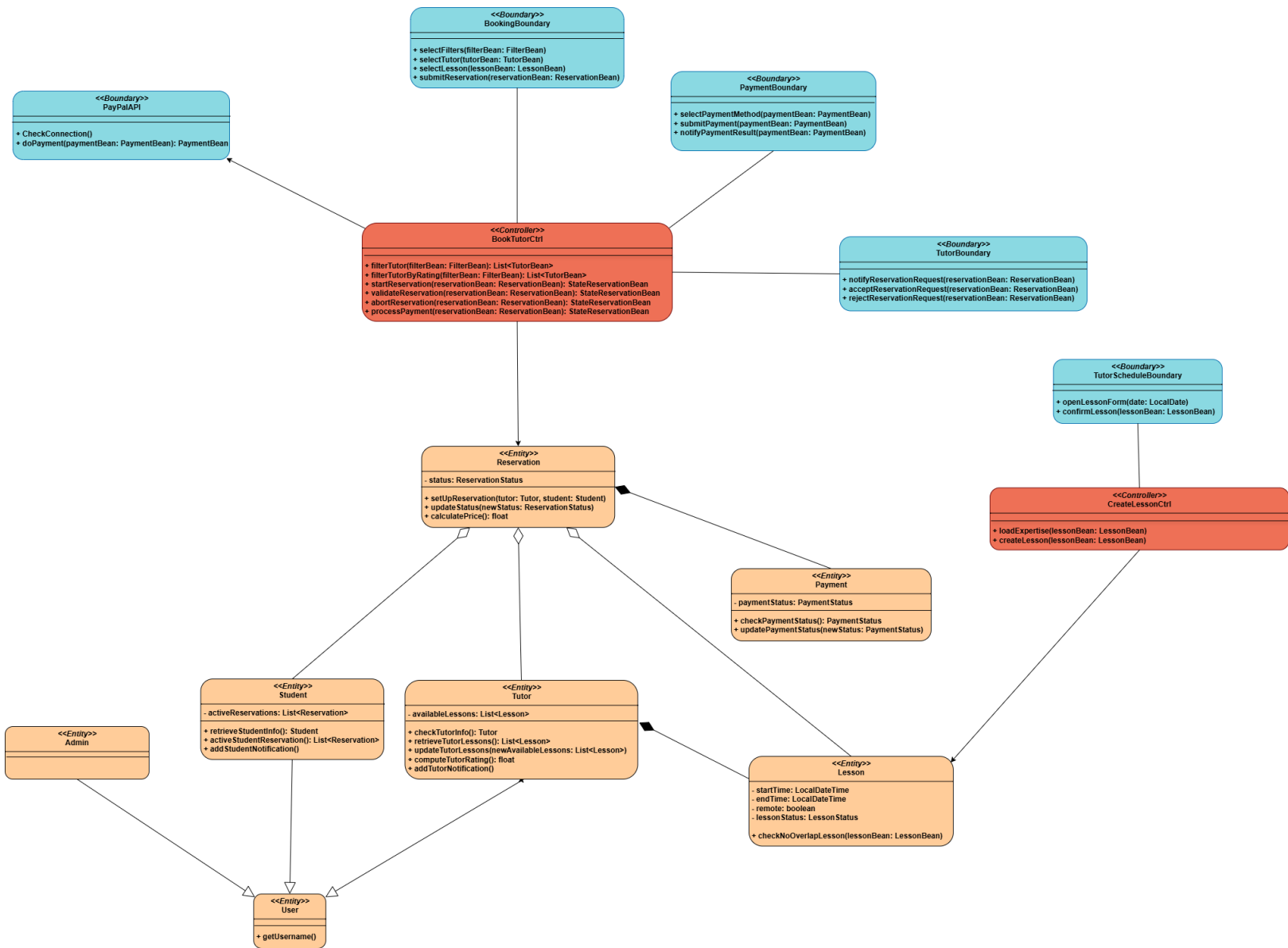
### 3.1 Class Diagram

The architectural design of Tutora adheres to standard software engineering patterns to ensure scalability, maintainability, and a clear separation of concerns. The system adopts the BCE (Boundary–Control–Entity) classification, which clearly separates user interface components, application logic, and domain data.

- Both main use cases include an authentication (login) step as a prerequisite for accessing system functionalities; however, for clarity and to avoid redundancy in the diagrams, the login process is explicitly represented only in the “Apply to become a tutor” use case.

#### UC-2: ApplyToBecomeATutor



**UC-1: BookTutor**

### 3.1.1 Design Patterns

The following section presents a series of diagrams illustrating the fundamental logical structure of the design patterns applied throughout our application.

Before proceeding to the visual models, there are two key aspects regarding our implementation to keep in mind:

- **Multiple Instantiations:** While the theoretical scope of the project outlines a total of six distinct design patterns (equally divided as three per developer), the actual implementation contains significantly more instances. Both developers have applied patterns of the same typology multiple times across the system.
- **Focus on Structure over Exhaustive Detail:** For practical purposes and visual clarity, the methods and attributes included within the diagrams represent only a partial view of the actual codebase. Our primary goal is to highlight the core architectural structure and the relationships defining each pattern, rather than focusing on exhaustive implementation details.





## Singleton Pattern

In Tutora, the Singleton pattern ensures that critical system components have a single, globally accessible instance throughout the application lifecycle.

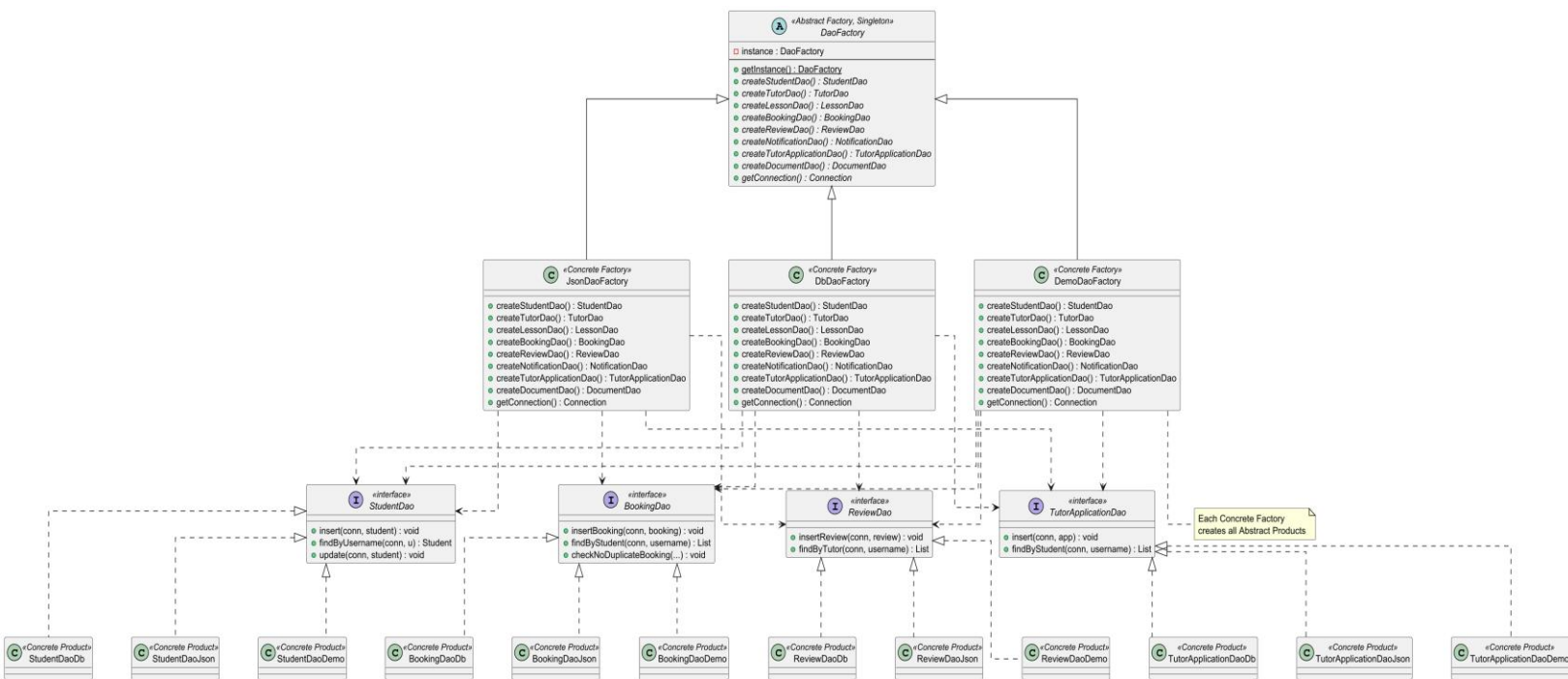
- **DAO Factory Management:** Applied in the *DaoFactory* class, which provides a single point of access to all Data Access Objects.
- **Session Handling:** Applied to the *SessionManager*, which manages active user sessions identified by unique tokens.
- **Scene Management:** Applied to the *SceneManager*, which controls JavaFX scene transitions and holds the current session token.



## Abstract Factory Pattern

The Abstract Factory pattern is used to manage the creation of the entire data access layer. The *DaoFactory* acts as an abstract interface, with three concrete implementations:

- **DatabaseDaoFactory**: creates DAOs that interact with a MySQL relational database.
- **DemoDaoFactory**: creates in-memory DAOs for demo mode (no external dependencies required).
- **JsonDaoFactory**: creates DAOs that persist data to JSON files, used for development and testing.

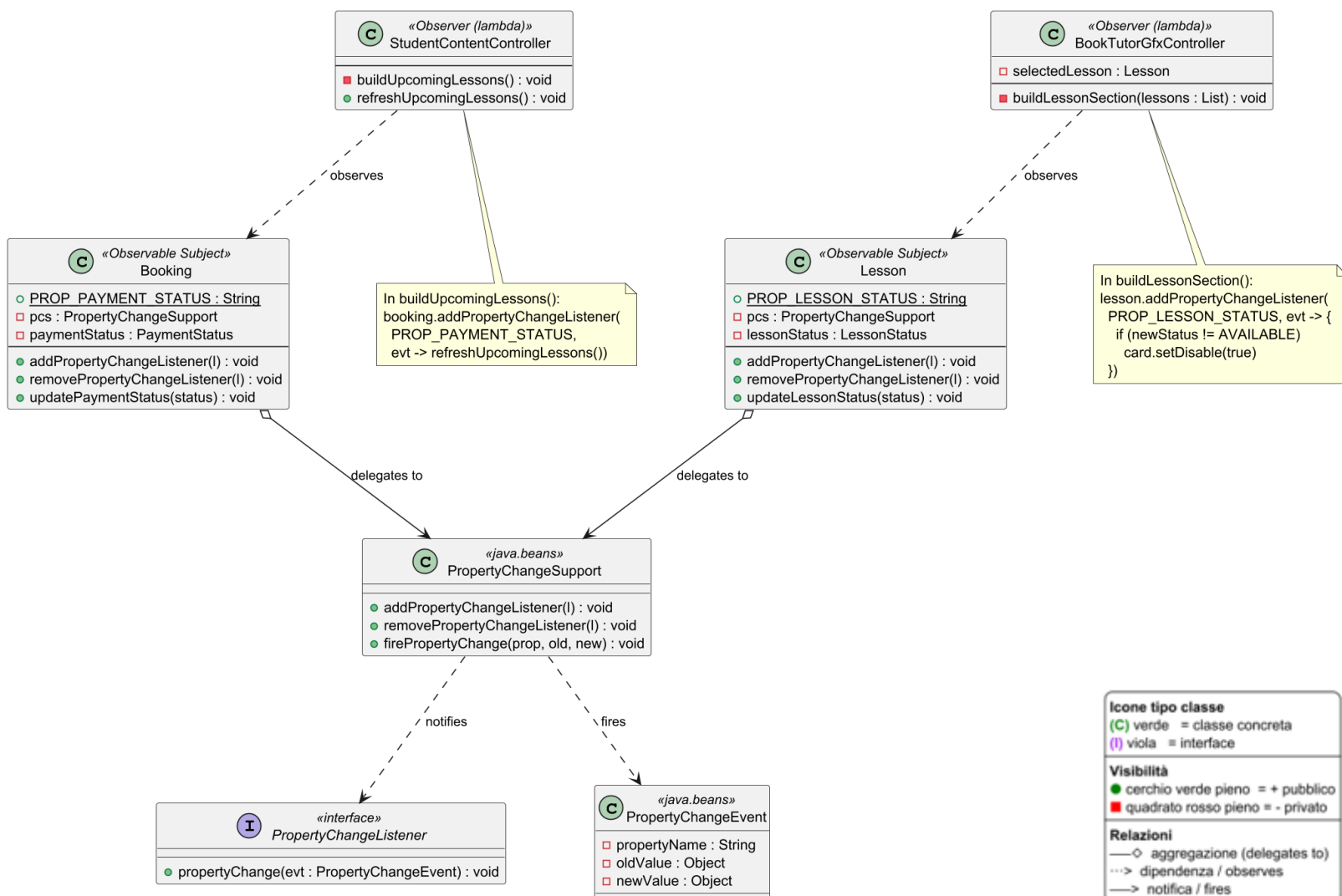


## Observer Pattern

The Observer pattern is used to propagate state changes from domain objects to the GUI without tight coupling.

- The observable classes (Booking and Lesson) use *PropertyChangeSupport* to notify listeners when relevant properties change.
- *StudentContentController* listens to Booking changes via `PROP_PAYMENT_STATUS` and automatically refreshes the upcoming lessons list when the payment status updates.
- *BookTutorGfxController* listens to Lesson changes via `PROP_LESSON_STATUS` and disables the corresponding booking card when a lesson becomes unavailable.

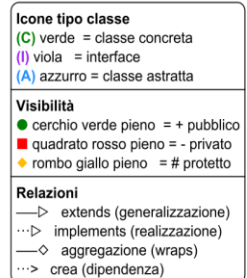
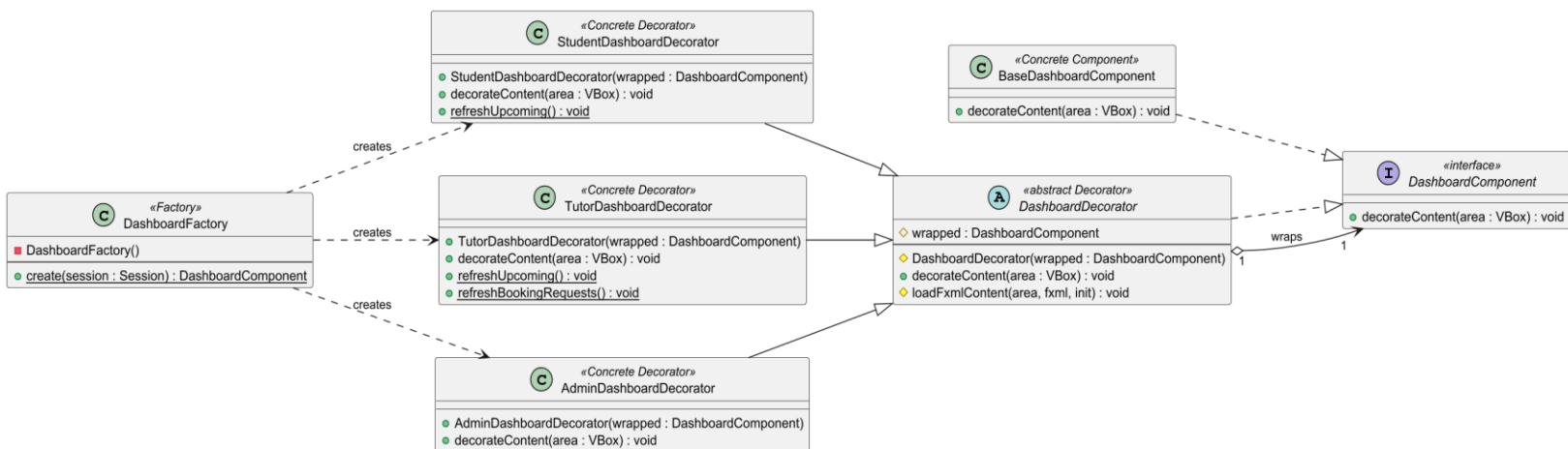
Both observers receive updated state through a **push model**, avoiding additional model queries. This approach improves extensibility, as new observers can be added without modifying the domain classes.



## Decorator Pattern

The Decorator pattern is used to build the home dashboard dynamically according to the authenticated user's role, avoiding complex conditional logic in a single view class.

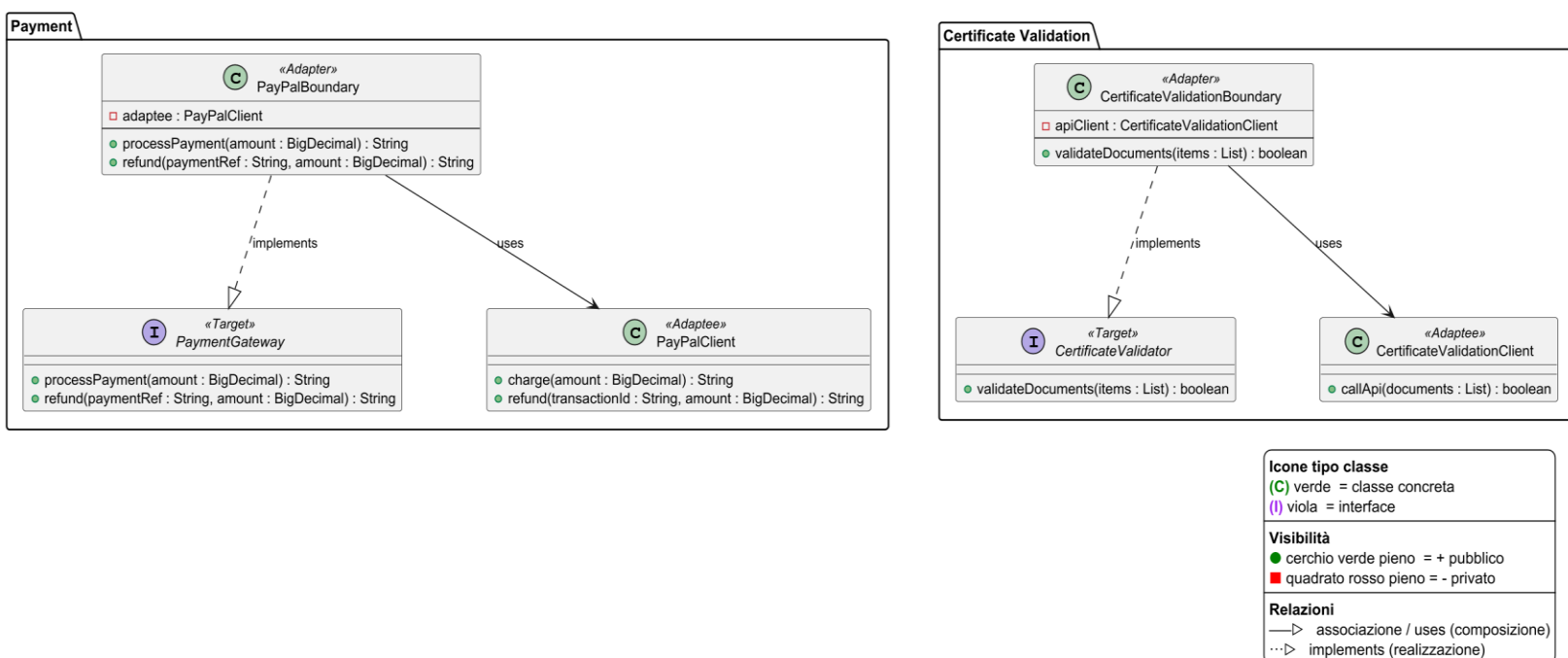
- *DashboardComponent* defines the common interface, while *BaseDashboardComponent* provides the empty base implementation.
- *DashboardDecorator* wraps another component and delegates content creation before adding role-specific sections.
- The concrete decorators *StudentDashboardDecorator*, *TutorDashboardDecorator*, and *AdminDashboardDecorator* extend the dashboard with the appropriate UI elements.
- *DashboardFactory* centralizes the assembly process by wrapping the base component with the correct decorator depending on the session role, making the system easily extensible for new roles.



## Adapter Pattern

The diagram below illustrates our application of the Adapter structural design pattern. We utilized the Object Adapter approach (relying on composition) to completely decouple our core business logic from external dependencies.

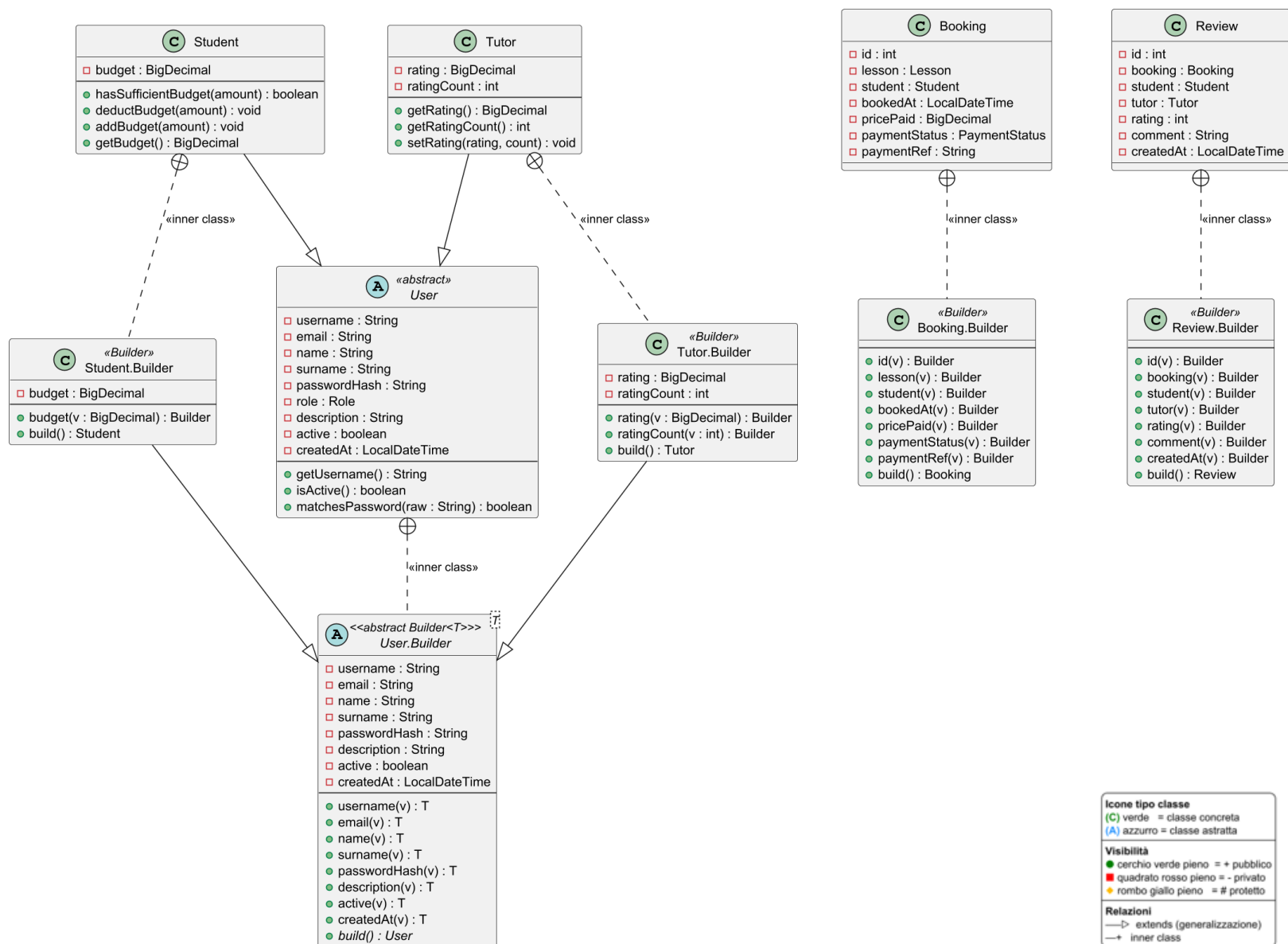
- In our current implementation, the external APIs (the *Adaptees*, such as *PayPalClient* and *CertificateValidationClient*) are simulated. We specifically chose the Adapter pattern to handle this exact scenario: the Adapters (or Boundaries) act as wrappers, bridging our standard system interfaces (the *Targets*) with these external services.
- Crucially, this architecture guarantees that when the real, production APIs are eventually integrated to replace the simulated ones, the core application logic will not need to be altered in any way. This approach ensures maximum flexibility and completely shields our system from future external changes.



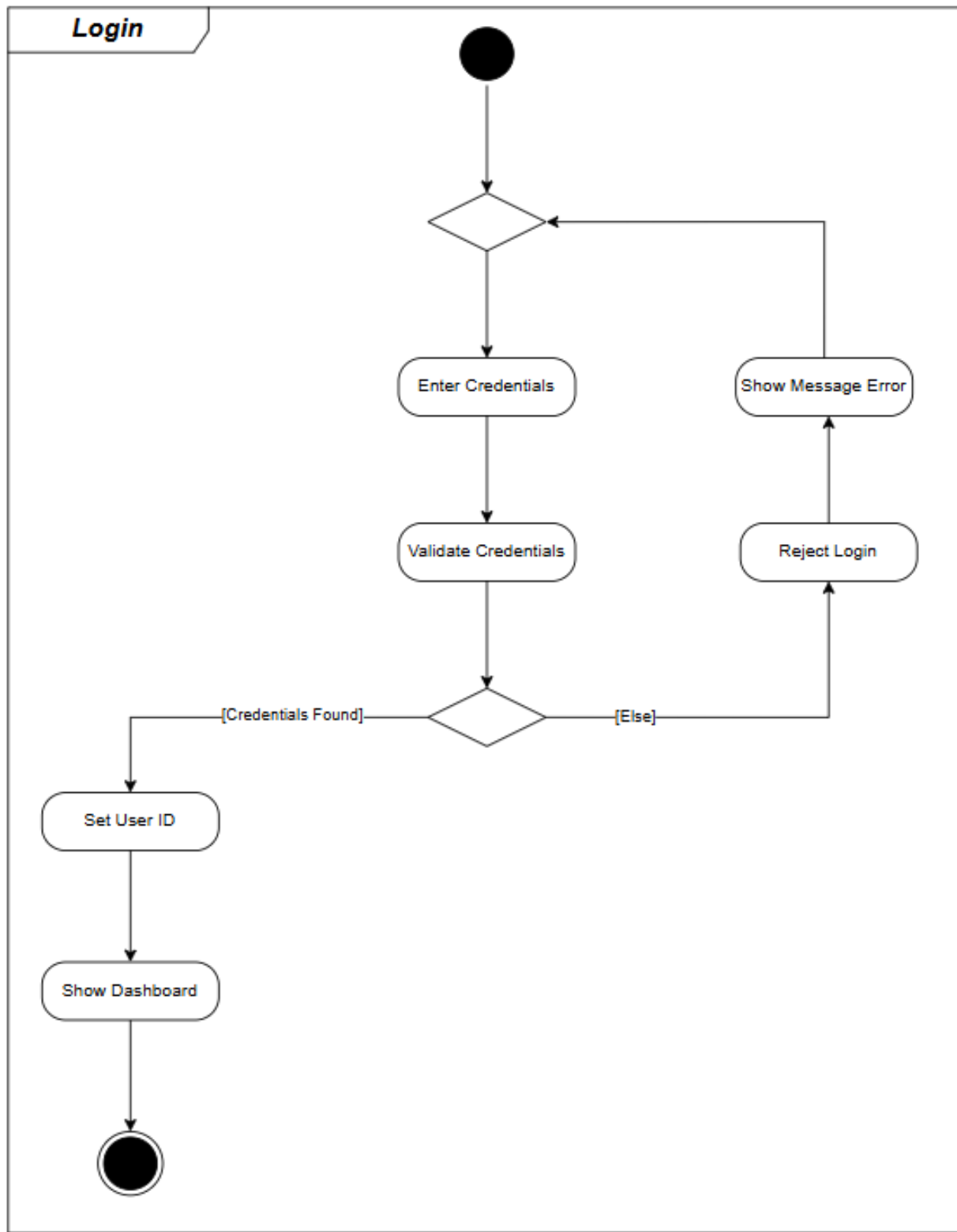
## Builder Pattern

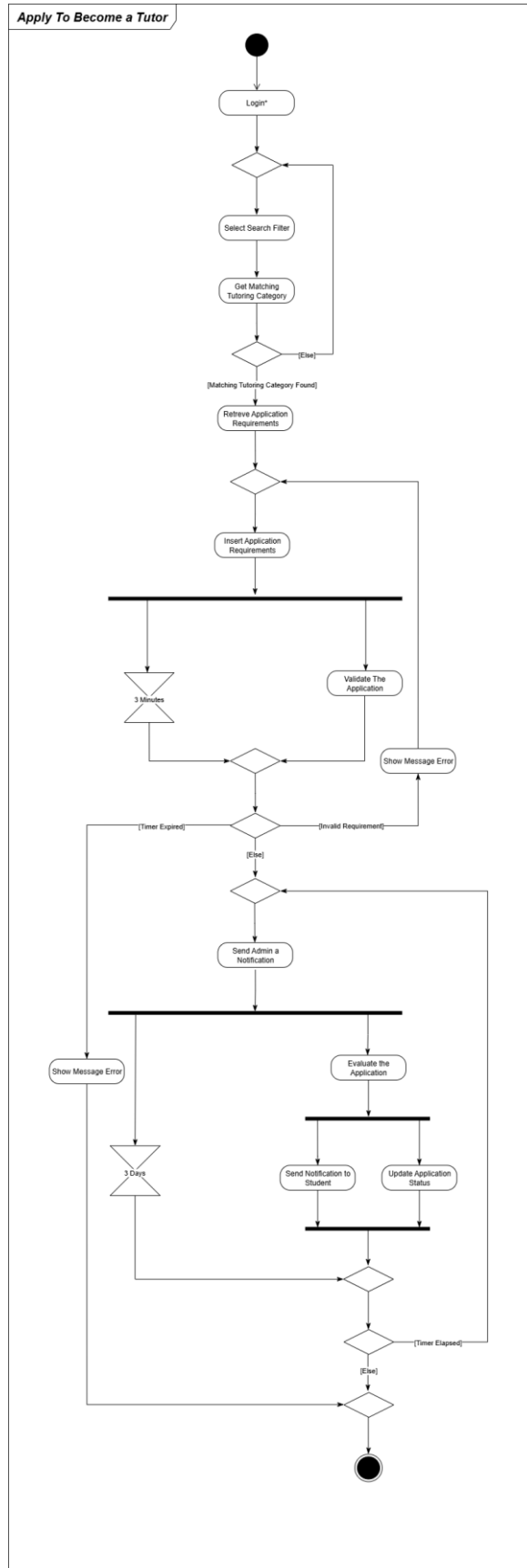
The Builder pattern is used to construct complex domain objects with multiple optional or shared attributes in a clear and maintainable way.

- Booking and Review provide static inner builders with fluent APIs, avoiding telescoping constructors and explicit null values for optional fields.
- The pattern is extended to the User hierarchy through an abstract generic *UserBuilder*, which encapsulates shared attributes such as username and email.
- Concrete builders (Student, Tutor, and Admin) extend this base builder to add role-specific fields, enabling readable, type-safe object creation while avoiding code duplication.

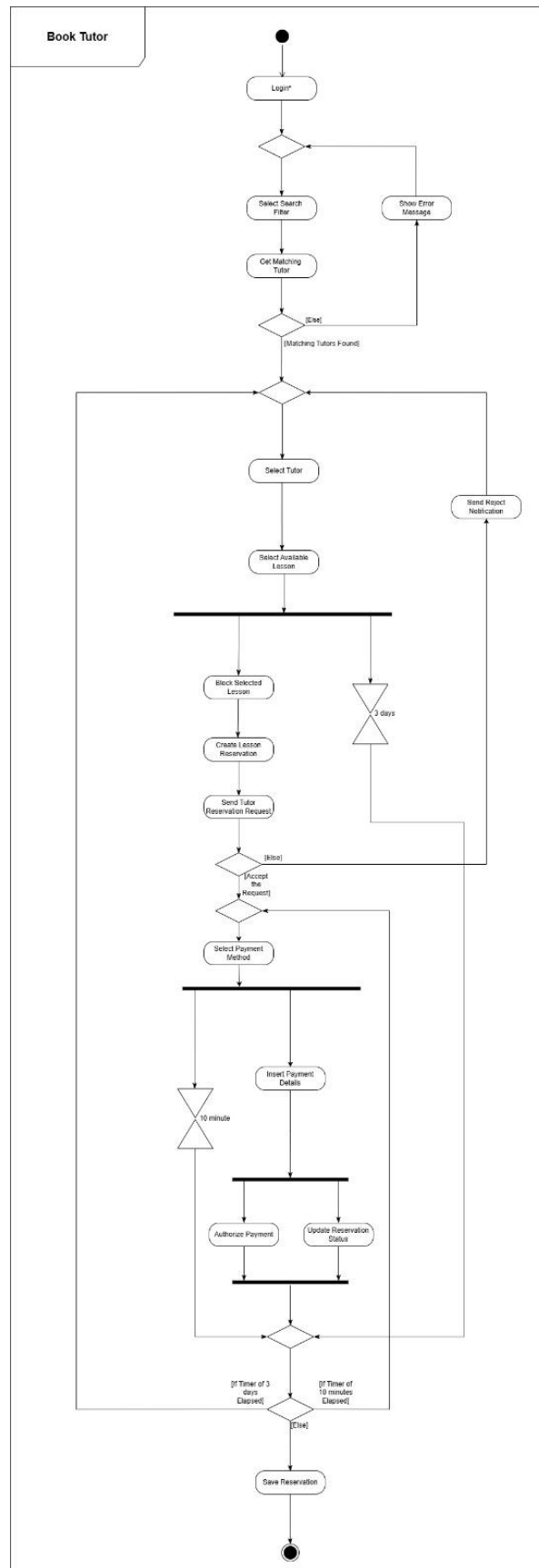


## 3.2 Activity Diagram

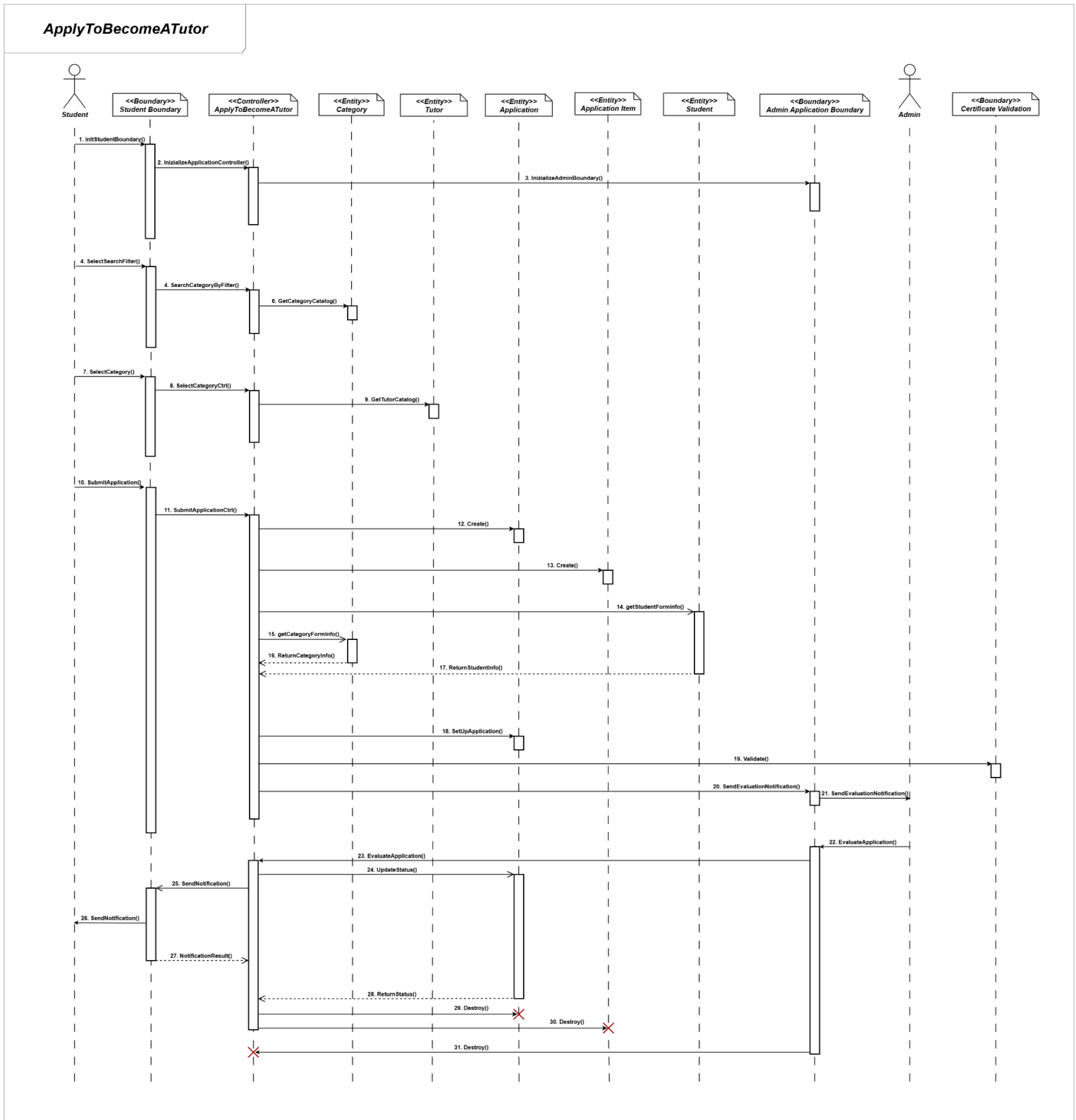


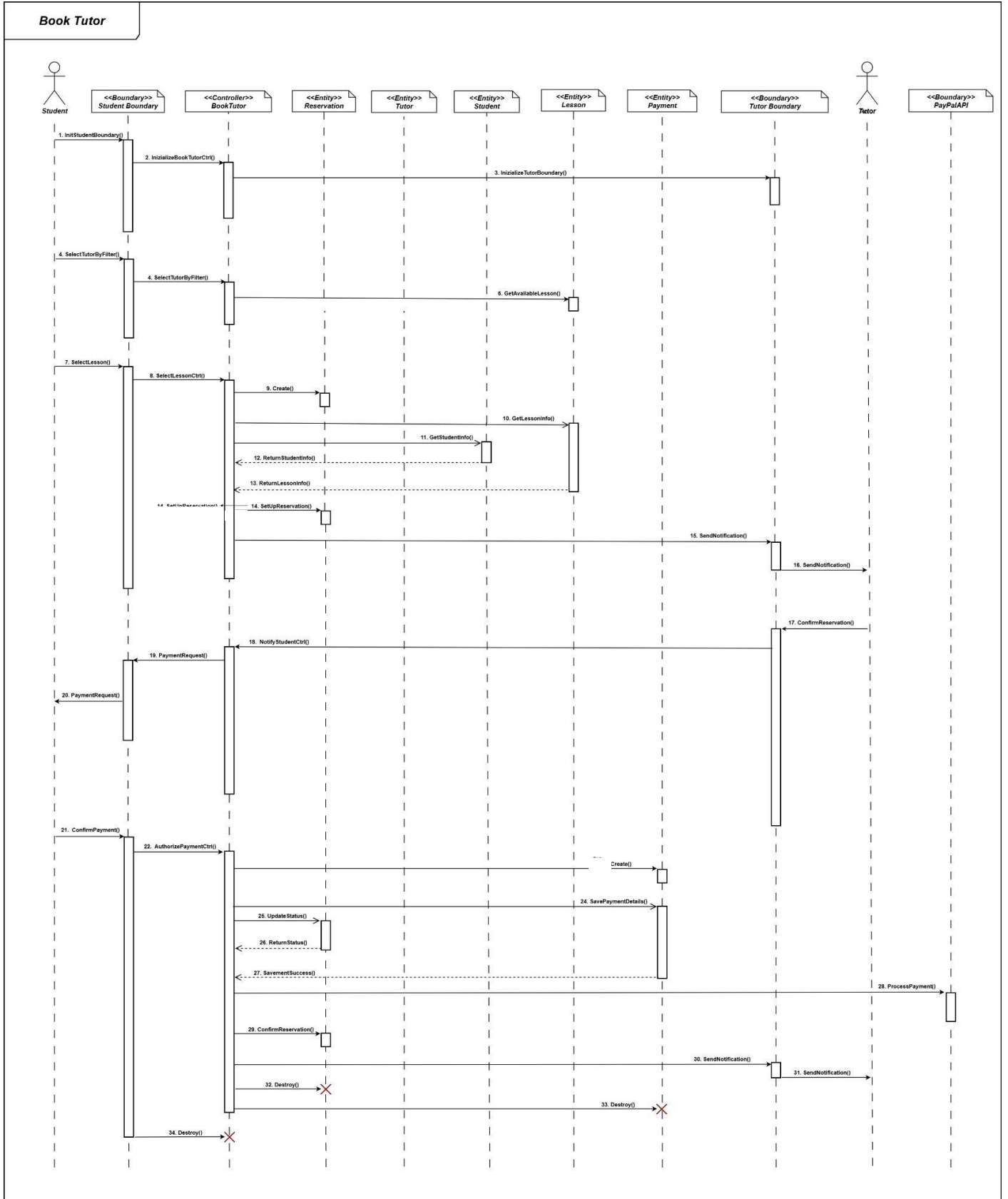






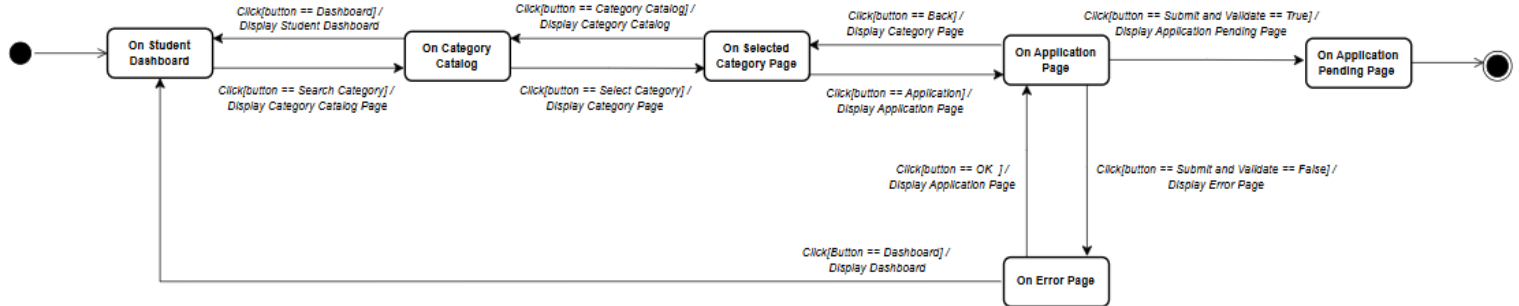
### 3.3 Sequence Diagram



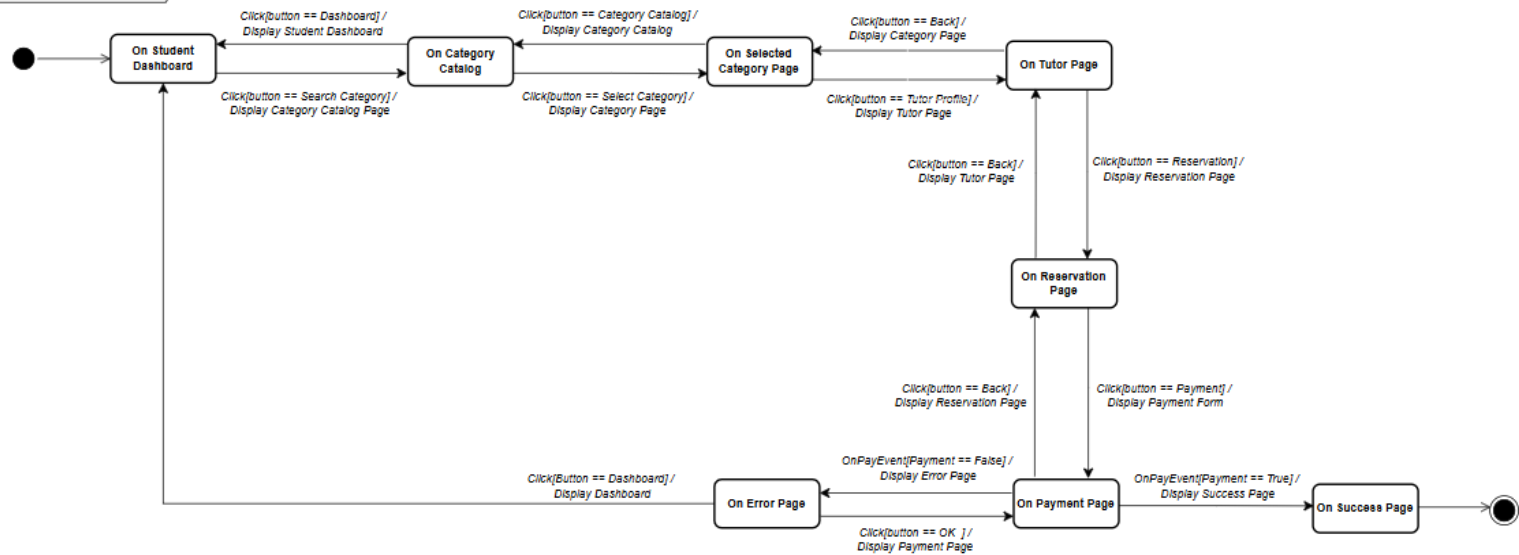


### 3.4 State Diagram

#### Apply to Become a Tutor



#### Book Tutor



## 4. Testing

---

### Booking Domain

#### ***Duplicate booking detection (Conflict case): Alessio Dainelli***

We tested the validation logic responsible for preventing overlapping lesson bookings. The tests verify that the system correctly identifies and blocks duplicate requests by raising a *DuplicateBookingException* when a student already holds an active paid booking for the same tutor and subcategory, ensuring data integrity.

#### ***Booking uniqueness validation (Success case): Alessio Dainelli***

We tested the specificity of the uniqueness constraints within the booking system. The tests verify that students are permitted to book lessons for different subcategories with the same tutor, confirming that the system accurately distinguishes between distinct services and does not over-block legitimate new bookings.

#### ***Student budget deduction: Alessio Dainelli***

We tested the arithmetic correctness of the payment module. The tests verify that the student's wallet balance is accurately reduced following a successful transaction, ensuring that the backend model maintains consistency between the initial budget and the final balance after a deduction.

#### ***Budget sufficiency guard (Insufficient funds): Alessio Dainelli***

We tested the pre-payment validation mechanism regarding financial coverage. The tests verify that the system prevents students with insufficient funds from completing a purchase, accurately returning a negative validation state when the required amount exceeds the available budget.

#### ***Budget boundary verification (Exact amount): Alessio Dainelli***

We tested the boundary conditions for student financial transactions. The tests confirm that the system correctly handles cases where the student's budget exactly matches the required fee, allowing the payment to proceed and verifying the precision of the comparison logic.

## Tutor Application Domain

### ***Tutor application duplicate check: Vittorio Iozzia***

We tested the persistence logic for prospective tutor applications. The tests verify that the system prevents the submission of redundant requests by raising a *DuplicateApplicationException* when a student attempts to create a second active application for the same subject category.

### ***Tutor application submission readiness: Vittorio Iozzia***

We tested the validation subsystem that evaluates the completeness of a tutor's profile. The tests verify that the system identifies missing or insufficient mandatory requirements—such as a motivation letter that does not meet the minimum character limit—effectively guarding against incomplete submissions.

### ***Tutor application status transition FSM: Vittorio Iozzia***

We tested the finite state machine (FSM) governing the lifecycle of a tutor application. The tests verify that illegal state transitions are strictly blocked, ensuring that applications cannot bypass mandatory steps, such as moving directly from a draft status to an accepted status.

### ***Review duplicate submission: Vittorio Iozzia***

We tested the integrity of the student feedback system. The tests verify that the data access layer raises a *DuplicateReviewException* if an attempt is made to insert a second review for the same completed booking, ensuring a strict one-to-one relationship between lessons and evaluations.

### ***Review rating validation: Vittorio Iozzia***

We tested the domain constraints applied to the qualitative data of reviews. The tests verify that the system enforces a valid rating range at construction time, ensuring that invalid inputs (such as a rating of zero) are rejected before being persisted in the database.

## Summary of Test Organization

The tests were organized into two test suites: *BookingDomain* (Alessio Dainelli), covering booking management and financial transactions, and *TutorApplicationDomain* (Vittorio Iozzia), covering tutor application lifecycles and student feedback.

## 5. Code

---

The complete source code is available at the following repository: [[GitHub Repository](#)]

## 6. Video

---

For a visual demonstration of the application's functionality, please refer to our YouTube video here: [[Watch the YouTube video here](#)]

## 7. Sonar Cloud

---

The code quality and static analysis have been monitored using SonarQube.

- **Project Key:** TUTORA
- **Dashboard:** [[SonarQube Dashboard URL](#)]